Assignment 3 — Cloud Computing and Microservices Resilience

AITU Digital Student Services Platform | Technical work report

Academic year 2026–2027 | Synthetic case study | Version 1.0

Submission note: This report uses fictional operational inputs from the supplied assignment. Experiment outputs are reproducible deterministic simulations, NOT observed Kubernetes cluster measurements. Personal fields must be completed by the student.

Submission information

Field	Entry
Student name	[enter your name]
Student ID / group	[complete]
Instructor / date	[complete]
Instructor-assigned variant	[complete if provided]
Repository / commit	[add after repository upload]
Work period	[record actual start and completion times]
Tools used	Python 3; python-docx; matplotlib; Kubernetes YAML; deterministic simulation model

Scenario and design requirements

The university platform serves 10,000 synthetic users with an assumed peak of 500 requests/second. Required service availability is at least 99.9%, maximum API latency is 500 ms, RTO at most 15 minutes and RPO at most 5 minutes. The architecture contains Load Balancer → API Gateway → Auth/Course/Registration → Database. Three application components and the database have two instances each.

Important scope limitation: neither the 500 req/s capacity nor 500 ms latency is established by the availability formulas. They require load testing of a deployed application; no such measurements are claimed here.

Part A — Architecture analysis (5 points)

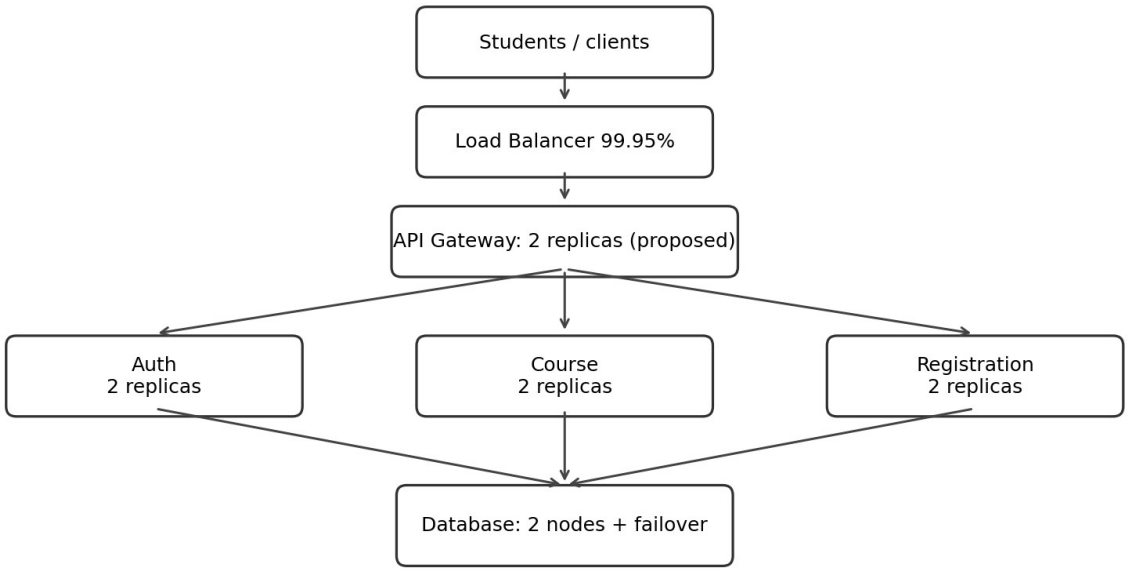


Figure 1. Proposed logical topology; two gateway replicas are the proposed improvement, not part of the baseline inputs.

Component	Example failure	Impact	Mitigation
Load Balancer	Process or network path fails	All traffic may be blocked	Health monitoring, standby load balancer / managed multi-zone routing
API Gateway	Single gateway crashes	All API endpoints unavailable	Introduce two gateway replicas behind load balancer
Auth	One authentication pod crashes	Login capacity reduces	Two replicas, readiness, restart, rolling deployment
Course	One course pod crashes	Course browsing slows or retries	Two replicas, readiness and idempotent retries
Registration	One registration pod crashes	Registration capacity drops	Two replicas, health checks, Service routing
Database	Primary node unavailable	State-changing requests fail until failover	Replicated DB, leader election, backups, tested recovery

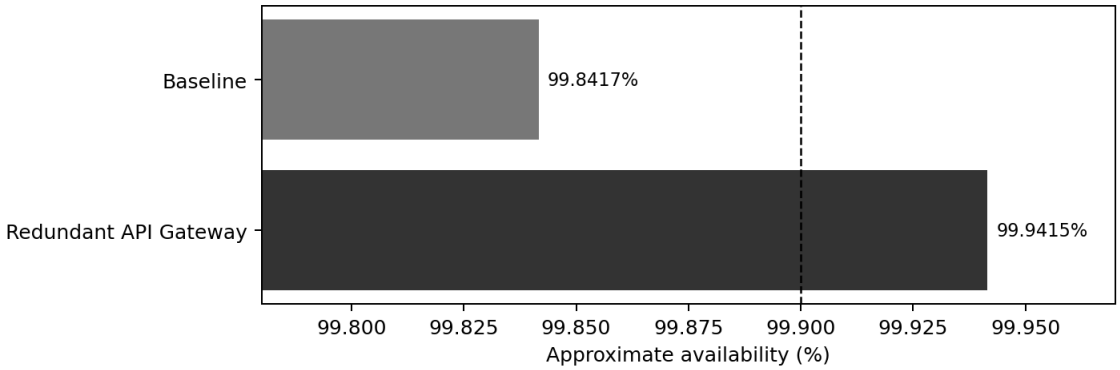
Architectural decision: duplicate API Gateway because its baseline 99.90% availability is the largest single-point bottleneck. Database redundancy only helps if failover and consistency are configured; the mathematical model assumes functioning failover.

Part B — Availability calculation (7 points)

For two independent identical units: Rparallel = 1 − (1 − R)^2. For sequentially required subsystems: Rseries = product of their availabilities. This is the requested simplified end-to-end model, not a full traffic-weighted multi-service SLO.

Subsystem	Configuration	Availability
Auth	2 × 99.50%	99.9975%

Course	2 × 99.50%	99.9975%
Registration	2 × 99.50%	99.9975%
Database	2 × 99.70%	99.9991%
Load Balancer	1 × 99.95%	99.9500%
API Gateway	1 × 99.90%	99.9000%
End-to-end baseline	$0.9995 \times 0.999 \times 0.999975^3 \times 0.999991$	99.8417%
Improved API Gateway	2 × 99.90%	99.9999%
Improved end-to-end	$0.9995 \times 0.999999 \times 0.999975^3 \times 0.999991$	99.9415%



Answer 1: baseline does NOT meet 99.9%; improved design meets the mathematical target. Answer 2: the gateway is the lowest-availability individual stage at 99.90%, followed by the load balancer at 99.95%. Answer 3: adding a redundant gateway yields about +0.09984 percentage points to the end-to-end model. The load balancer remains the next major risk.

Assumptions: independent failures; no correlated zone, deployment or configuration incidents; replicas adequately provisioned and failover correctly wired; every stage required for the modeled request. These assumptions can overstate production availability.

Part C — Kubernetes resilience design (7 points)

Configuration reference: config/registration-deployment.yaml and config/namespace.yaml. Application reference: config/registration-demo-app.py and config/Dockerfile. The image registration-demo:1.0.0 is locally buildable and must be made accessible to the Kubernetes cluster; it is not a published image.

Mechanism	Implemented?	Purpose / setting
Replicas	Yes	Deployment replicas: 2; tolerates one replica loss
Liveness	Yes	GET /healthz; 10 s period; 3 failed checks triggers kubelet restart
Readiness	Yes	GET /readyz; 5 s period; 2 failures excludes unhealthy pod
Resources	Yes	100m/128Mi requests, 500m/256Mi limits per pod
Rolling update	Yes	maxSurge: 1, maxUnavailable: 0
Automatic restart	Yes	Deployment pod restart via kubelet and controller

		reconciliation
Service discovery	Yes	ClusterIP registration.aitu-demo.svc.cluster.local

Chosen tuning: maxUnavailable: 0 avoids intentionally reducing the two available replicas during rollout. A readiness probe that depends on database availability removes unhealthy backends from routing; during a full DB outage this means all endpoints may become unready. This is deliberate fail-closed behavior for this demonstration.

Validation constraints: YAML can be parsed offline, but readiness behavior, pod scheduling, rolling update and 500 req/s performance require a running cluster. No real AITU endpoint or credential is used.

Part D — Failure simulations (6 points)

Method: run python experiments/run_simulations.py. Outputs are written to experiments/evidence. This is a deterministic event-timeline MODEL, not empirical Kubernetes telemetry. All timing assumptions are documented in that script; do not submit the traces as authentic pod logs.

Experiment	Failure injected	Detection	Recovery	Student impact
1	One Registration replica fails	10 s (model)	20 s (model)	Second pod serves requests; less capacity and possible brief errors
2	Both Registration replicas fail	10 s (model)	25 s (model)	Registration unavailable until first new pod ready
3	Database unavailable	10 s (model)	185 s (model)	DB-dependent requests fail; registration endpoints unready

Interpretation: a single pod fault can be masked by a healthy replica. Both replica failures create a true service outage; the controller must create functioning replacements. Database failure defeats service replication unless the database itself has an effective failover mechanism.

Model limitation: chosen detection/recovery durations are scenario assumptions rather than measured cluster performance. A real experiment should record injection timestamp, Kubernetes events, kubectl get pods -w output, pod health checks, request status and recovery timestamp. Example commands are included in README.md.

Part E — RPO/RTO and disaster recovery (5 points)

Mechanism	Frequency	Target RPO	Estimated recovery	Purpose
Database log shipping / PITR	Continuous; lag monitored < 5 min	≤ 5 min if replication and backups validated	Primary failover target ≤ 3 min	Minimize lost committed transactions
Database base backup	Daily; restore tested weekly	Alone: up to 24 h (insufficient)	Restore varies; must test	Provides recoverable base image; pair with

				WAL/log shipping
Replica configuration backup	On configuration change + daily export	Config state ≤ 24 h; not transactional RPO	Target ≤ 10 min	Recreate Deployment/Service manifests
Full disaster recovery	Monthly drill / restore verification	≤ 5 min design target	≤ 15 min design target	Restore data, manifests, networking and validate service

High availability reduces the impact of routine instance failures with redundancy and automatic failover; disaster recovery rebuilds service and state after larger outages, deletion or corruption. A replica is not a substitute for offline/immutable backups because deleted or corrupted data may replicate too.

RTO and RPO above are target engineering plans, not experimentally verified service-level guarantees. Proving them requires full restoration drills and monitoring replication lag. In the given model, a 185-second DB outage is below the 15-minute RTO target, but does not establish the RPO.

Final architecture decision and personal analysis

The highest-leverage initial change is to make API Gateway redundant. Under independent-component assumptions this increases modeled availability from 99.8417% to 99.9415% (+0.09984 percentage points), crossing the 99.9% target. This does not remove load-balancer, common-mode and database failover risks.

One limitation: the simple series model assumes every request needs Auth, Course and Registration in sequence, while real user journeys may call only one microservice. A better production estimate would use per-journey SLOs plus dependency and traffic traces.

Individual work statement: replace this paragraph with details of what you personally executed, the YAML option you changed, the obstacle you actually encountered and how you resolved it. Do not claim to have operated a Kubernetes cluster unless you have done so.

Evidence index and reproducibility

Evidence item	Path
Architecture drawing	diagrams/architecture.png
Availability chart	diagrams/availability_comparison.png
Kubernetes workload and Service	config/registration-deployment.yaml
Namespace	config/namespace.yaml
Minimal runnable demo service	config/registration-demo-app.py, config/Dockerfile
Simulation source	experiments/run_simulations.py
Machine-readable model results	experiments/evidence/simulated_results.csv and .json
Per-scenario event traces	experiments/evidence/model_trace_1.txt through _3.txt
Execution and verification guide	README.md

Oral defense — compact explanation

We modeled a student-services platform with redundant microservices. With two identical independent instances, subsystem uptime is $1 - (1 - R)^2$. The original topology gets 99.8417%, so it misses the 99.9% requirement. The gateway is the dominant single point of failure, and redundancy raises modeled availability to 99.9415%. Kubernetes keeps two Registration pods, probes them for health, updates without planned loss of available pods and exposes a stable Service name. Our included experiments are deterministic simulations: one pod failure is masked, both pod failures cause temporary loss, and a DB outage affects requests despite app-level replicas. HA is not DR; for disasters we need transaction-log recovery, backed-up state and tested restoration within RTO/RPO targets.

Final checklist

Completed: architecture/table; calculations; YAML/demo code; explicitly labeled simulation model/results; RPO/RTO strategy; final rationale; defense notes. Must be completed by student: identity, assigned variant, real work timestamps, actual Kubernetes deployment screenshots/logs if instructor requires real-cluster evidence, actual load/latency testing, personal incident and signature.

Academic integrity note: all operational platform data are synthetic per the assignment, and every modeled observation is clearly labeled. No real AITU system, real user, credentials or infrastructure was accessed.